APRESENTAÇÃO DE PROJETO · 21.10.2026

projetoFans

O que a turma precisa saber sobre a semana — provas, entregas e o material que caiu em cada aula.

FANS · NOVA SERRANA, MG

Otávio de Pádua Rodrigues Ribeiro

Engenharia de Software · 6º período · 2026

Trabalho Interdisciplinar I

Prof. César Augusto Oliveira Soares

Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas

Prof. Daniel Lucas de Souza Lelis

01 · O PROBLEMA

Quem perde a prova raramente perde por falta de aviso.

o aviso afunda

O grupo tem 400 mensagens por semana

A data foi dita na terça, entre um meme e uma dúvida de outra matéria. Na quinta ninguém acha.

quem faltou não recupera

O material fica com quem foi

Pedir a foto do quadro no grupo é pedir favor. E favor depende de alguém estar com disposição.

ninguém é dono

Todo mundo acha que alguém anotou

A planilha que uma pessoa mantinha no primeiro período morreu no segundo. Sempre morre.

A informação existe. O que falta é um lugar onde ela não afunde.

02 · OBJETIVO E PÚBLICO-ALVO

Um lugar onde a turma mantém, *junta*, o que precisa saber da semana.

OBJETIVO

Responder duas perguntas em menos de cinco segundos, sem depender de ninguém responder no grupo:

- **O que vem aí?** — provas e entregas dos próximos 14 dias, por urgência.
- **O que foi dado?** — o material de cada aula, por matéria, com quem subiu.

A proposta de valor é estreita de propósito: o app vende **“o que foi dado no período”**. Nunca “o que vai cair na prova” — isso seria promessa que ele não pode cumprir.

PÚBLICO-ALVO

Uma turma de graduação que já se conhece

De 20 a 60 alunos do mesmo período, com matérias em comum e um grupo de mensagens que já existe.

Esse recorte é o que permite não ter moderação prévia: **qualquer um posta, sem aprovação**, porque todos sabem quem é quem.

Secundário: o professor, como contribuidor **opcional**.

03 · ESCOPO

Não é um Google Classroom.

Não tem envio de trabalho, não tem correção e não tem nota. Professor é contribuidor **opcional** — turma sem professor funciona exatamente igual.

- assinatura
- resumo por IA
- notas e boletim
- integração com a faculdade
- múltiplas turmas

Cada um desses foi recusado por escrito, com data. É por isso que o que existe funciona.

04 · O SISTEMA FUNCIONANDO

Demonstração ao vivo

No ar desde setembro, em celular de verdade, com dados de verdade — e com gente da turma já usando.

01

A tela inicial

Próximos 14 dias, por urgência, com o nome de quem cadastrou.

02

Cadastrar uma prova

Sem pedir autorização a ninguém. Aparece para a turma na hora.

03

Material da aula

Vinculado à prova: a resposta de “o que eu estudo”.

04

Marcar como resolvido

Só meu. Ninguém mais vê — e isso é garantido pelo banco.

05 · PRINCIPAIS FUNCIONALIDADES

Quatro telas, e uma regra em cada uma.

Início — o que vem aí

Provas e entregas dos próximos 14 dias, teto de 6 itens, ordem por urgência. **A janela, o teto e a ordem são calculados no servidor**, nunca no celular — assim existe uma implementação e uma suíte de testes, em vez de cada tela divergir.

Matérias — o que foi dado

Material de **aula** e material **adicional**, separados. Link ou arquivo. Na tela de uma prova aparecem só os materiais vinculados àquela prova — nunca o semestre inteiro.

Calendário — o mês

Grade mensal que anda para frente e para trás, com marca por matéria no dia. Toca no dia e vê o que tem nele. Mais as anotações da turma.

Meu quadro — só meu

A fazer e feito. **Estado pessoal vive em outra tabela**, então marcar resolvido não altera uma linha do registro compartilhado. A regra é garantida pelo banco, não por disciplina de quem programa.

Mais: lembrete na véspera, convite por código de 8 caracteres, e histórico de quem editou o quê.

06 · ARQUITETURA

O celular é vitrine. A regra toda vive no servidor.

cliente

App Android

Ionic 8 + Angular 20, componentes standalone e signals. Empacotado com Capacitor.

Cache local para funcionar sem rede e para agendar o aviso da véspera.

HTTPS
JWT

api rest · 16 controllers

Spring Boot 3.5

Java 17. Camadas: controller → service → repository. Entidade do banco **nunca** cruza o controller.

Hospedada no Render.

JDBC

persistência

PostgreSQL 16

Schema versionado em 10 migrations Flyway. Migration aplicada nunca se edita: cria a próxima.

Hospedado no Neon.

Escopo de turma num lugar só. Nenhum controller filtra turma à mão — tudo passa por um componente único. É o arquivo que se mudaria para suportar várias instituições.

O aviso da véspera é local. O servidor avisa o aparelho quando algo muda; o alarme é agendado no celular. Assim servidor dormindo não derruba a notificação.

07 · NÍVEIS DE ACESSO

Quatro papéis — e nenhum deles edita o registro de outra pessoa.

Papel	Pode	Não pode
Aluno	Criar prova, entrega, matéria, material e anotação. Editar e excluir **o que ele mesmo criou**.	Tocar em registro de outra pessoa. Ver dado de outra turma.
Líder	Tudo do aluno, mais **ocultar da turma** um registro já publicado e pedir que o autor o exclua.	Editar conteúdo alheio. Impedir alguém de publicar. Ele muda visibilidade, nunca o que está escrito.
Professor / Coordenador	Contribuir como os alunos, com selo de docente. Coordenador alcança as turmas sob sua coordenação.	Nada, enquanto a conta não for **aprovada** por um administrador. O papel sozinho não concede poder.
Administrador	Aprovar ou recusar conta de docente e vinculá-la a turmas.	Ler conteúdo de turma de que não participa.

Entrar na turma exige o código de convite. Criar conta é livre; entrar na sala não. O código é a fronteira de confiança — não o provedor de login.

Professor é opcional. Uma turma sem nenhum docente funciona igual. Dar poder obrigatório ao professor o tornaria requisito de funcionamento.

08 · SEGURANÇA APLICADA

Toda checagem acontece no servidor. Nenhuma só na tela.

identidade

Senha com BCrypt, sessão com JWT

Login por senha ou Google. O token do Google é validado no servidor e trocado por um JWT próprio — a API conhece uma forma só de autenticação.

isolamento

Registro de outra turma responde 404

Não 403. Um “proibido” confirmaria que o registro existe — o próprio status vazaria informação. Todas as rotas passam pelo mesmo resolvedor de turma.

edição simultânea

Nada é sobrescrito em silêncio

Toda alteração manda a versão que leu. Se outra pessoa salvou nesse meio-tempo, a API recusa com 409 e a tela recarrega — em vez de a edição de alguém sumir.

histórico

Editar não apaga o que havia

Toda edição grava o estado anterior numa tabela de revisões. Exclusão é sempre lógica: o registro sai da lista, não do banco.

força bruta

Teto de tentativas de login

E trocar a senha derruba as sessões abertas — senão trocar a senha depois de um vazamento não expulsaria ninguém.

exposição

A senha nunca sai do banco

A entidade do banco não cruza o controller: a API monta um objeto de resposta à mão. É o que impede o hash de vazar numa serialização automática.

09 · AUDITORIA DE SEGURANÇA

Três falhas reais foram encontradas no próprio projeto. E provadas.

achado 1 · corrigido

Tomada de conta pelo login do Google

O sistema usava o e-mail do token para decidir que dois logins eram a mesma pessoa, sem conferir se o e-mail era **verificado**. Um domínio corporativo pode emitir token com e-mail não confirmado — e quem o apresentasse assumia a conta da vítima.

achado 2 · corrigido

Registro oculto respondia pelo id direto

O líder ocultava uma prova, mas quem soubesse o id continuava lendo o registro inteiro — e os ids já estavam em todo aparelho que havia sincronizado. A moderação furava por baixo.

achado 3 · corrigido

Qualquer um desvinculava material alheio

Dava para arrancar todos os materiais da prova de outra pessoa — e sumia sem rastro, porque o vínculo era apagado de verdade e o histórico não cobria esse caso.

Verificadas por mutação, não por teste verde

Teste que passa não prova que a proteção funciona — prova que ele não a exercita. Para cada falha eu **desliguei a guarda** e confirmei que o teste ficava vermelho. Só então religuei. As três eram exploráveis de verdade.

10 · OBSTÁCULOS

As três dificuldades que mais custaram tempo.

27 d

O Docker da minha máquina morreu — e com ele, a suíte de testes

Os testes de banco dependiam de um contêiner Docker. O serviço parou de subir em 3 de agosto e não voltou. Por **27 dias nenhuma tarefa de backend conseguiu atingir o próprio critério de aceite**, porque o critério era “os testes passam” e eles não rodavam.

Solução: troquei por um Postgres **embutido** — binário real do banco, baixado pelo próprio teste, sem precisar de serviço nenhum rodando. Recusei usar um banco “leve e quase compatível”: o schema usa recursos específicos do Postgres, e um banco que mente num teste é pior que um teste que não roda.

2 JDK

O mesmo computador precisa de duas versões de Java

O servidor compila com Java 17; o empacotamento Android exige 21 ou mais. Apontar a variável de ambiente para um quebra o outro — e quebrou os dois lados, em semanas diferentes, com a mesma mensagem enganosa.

Solução: documentei qual versão cada comando exige, com a mensagem de erro exata que aparece quando está errada. Problema de ambiente não se resolve com código: se resolve escrevendo.

0

Um bug que não existia

Um teste da interface acusava falha num código que, lido com atenção, tornava aquela falha impossível. Quase refiz a funcionalidade inteira.

Causa: o cache de compilação estava servindo a versão antiga. Fonte e execução discordando. Hoje é a primeira coisa da lista de verificação — custa 40 segundos e já ia custando uma tarefa refeita.

11 · RECURSOS UTILIZADOS

Cada escolha tem um motivo escrito.

Camada	Tecnologia	Por que esta
Linguagem	**Java 17** · Spring Boot 3.5.3	Tipagem forte e ecossistema maduro para a camada que guarda os dados da turma.
Banco de dados	**PostgreSQL 16**	O modelo tem relação muitos-para-muitos e histórico auditável. Banco de documentos atrapalharia exatamente isso.
Versão do schema	**Flyway** · 10 migrations	O banco evolui por arquivos numerados e versionados. Migration já aplicada nunca se edita.
Aplicativo	**Ionic 8 + Angular 20**	Um código só para Android e navegador. Componentes standalone e signals.
Empacotamento	**Capacitor 8.5**	Transforma o app em APK instalável, com acesso a notificação local.
Testes	**JUnit 5** · Postgres embutido · Karma	Testes de servidor rodam contra um Postgres de verdade, sem exigir serviço instalado.
Hospedagem	**Render** (API) · **Neon** (banco)	Plano gratuito. O app foi desenhado assumindo que o servidor hiberna.
Produtividade	**Git/GitHub** · Figma · Claude Code	Versionamento, protótipo de telas e programação assistida — com diário de decisões no repositório.

12 · O PROJETO EM NÚMEROS

Julho a outubro de 2026.

413

testes automatizados no servidor

em 51 arquivos, todos verdes

48

suítes de teste no aplicativo

interface e camada de dados

26k

linhas de código

10 mil de Java, 16 mil de TypeScript

38

entradas no diário de decisões

cada uma datada, com o que travou

16 endpoints agrupados por recurso — autenticação, turma, matéria, material, evento, quadro, moderação, anotação, docente, administração, sincronização.

12 cores de matéria, todas com contraste verificado por cálculo — não escolhidas no olho.

19 verificações manuais com duas contas reais contra o servidor no ar, em setembro.

13 · BENEFÍCIOS PÓS-IMPLANTAÇÃO

O que muda para quem usa — e já mudou.

Para o aluno

- Consulta em vez de rolagem: a data da prova está num lugar fixo, não enterrada no grupo.
- Quem faltou recupera o material da aula sem pedir favor a ninguém.
- Aviso na véspera, no celular — e ele funciona mesmo com o servidor dormindo.
- “O que eu estudo” tem resposta: o material vinculado àquela prova.

Para a turma

- A informação deixa de morrer junto com a pessoa que mantinha a planilha.
- Qualquer um alimenta, então não depende de ninguém em particular estar disposto.
- Autoria visível em tudo: dá para saber de quem veio a informação e quando.
- Erro se conserta, e o conserto fica registrado.

Para o professor

- Um canal para divulgar material, **sem virar obrigação**.
- Com selo de docente, para a turma distinguir a fonte.
- Nenhum trabalho novo: a turma funciona igual se ele nunca entrar.

Já implantado: no ar desde setembro, com colegas da turma usando em celular próprio.

O que ainda não está resolvido.

LIMITES CONHECIDOS

Isolamento só na aplicação. Todas as rotas filtram por turma corretamente, mas o banco não tem uma segunda barreira própria. Uma rota nova escrita sem cuidado não seria barrada duas vezes.

Força bruta. Há teto de tentativas, mas o limite prático ainda é o hardware do plano gratuito.

Sem política de conteúdo no navegador. Não há falha hoje, mas é defesa em profundidade que falta.

iOS não foi testado. O código compila, mas o comportamento de notificação em segundo plano é diferente lá.

PRÓXIMOS PASSOS

Envio de arquivo de verdade. Hoje material é link. O armazenamento está desenhado e falta ligar — e é provável que foto do quadro seja o uso principal.

Aviso quando o colega cadastra. Hoje o lembrete da véspera é local. Falta o empurrão imediato quando alguém publica algo novo.

Segunda barreira no banco. Fechar o limite de isolamento acima.

Distribuição. Hoje é APK por link. Publicar na loja é o passo para sair da turma piloto.

15 · ENCERRAMENTO

Perguntas?

E se alguém quiser testar, o sistema está no ar agora — não é protótipo.

TESTAR O APP

Instale o APK e entre com o código da turma de demonstração:

65722528

Turma separada, criada para esta apresentação — os dados da minha turma não ficam expostos.

SUGESTÕES E MELHORIAS

Abra uma issue no repositório. Toda sugestão vira item com data e resposta:

github.com/GODsGhst/Projeto-Fans/issues

Dentro do app também há um caminho para relatar problema.

O QUE EU QUERO OUVIR

- Alguma tela ficou parada ou em branco?
- Algum botão não fez nada?
- Alguma mensagem deixou dúvida sobre o que aconteceu?
- O que faltou para você usar de verdade?